

MSDM 5001 – Introduction to Computational and Modeling Tools

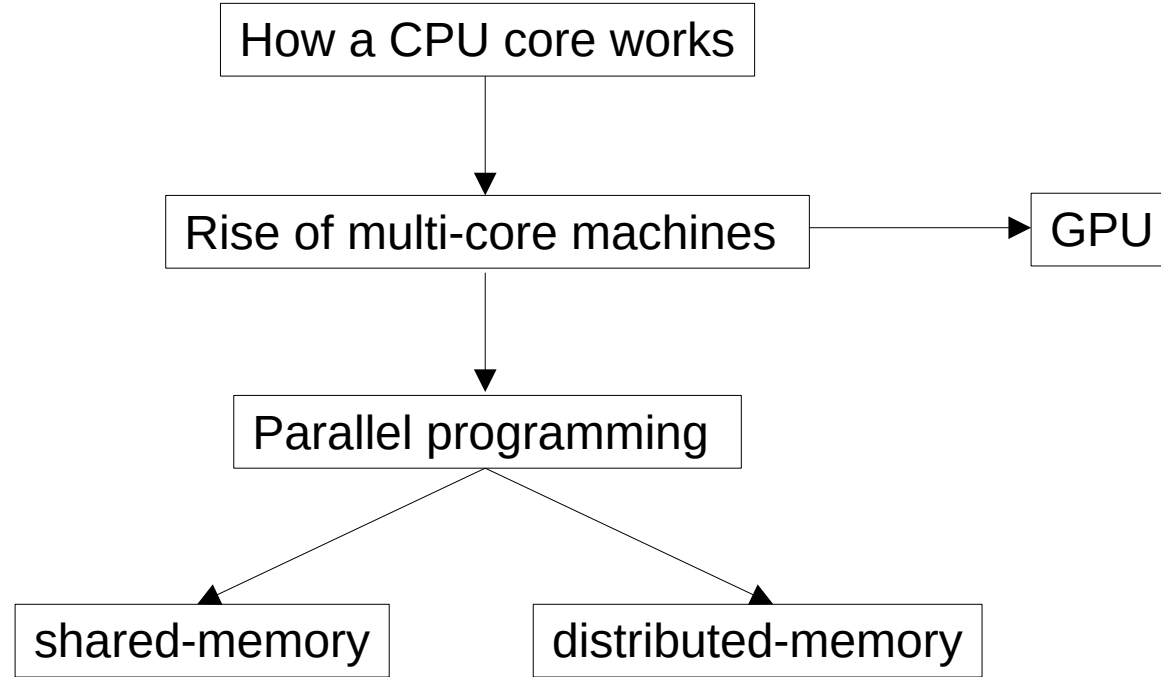
Quiz 1 Review

Computer Architecture + Shared-memory parallel programming

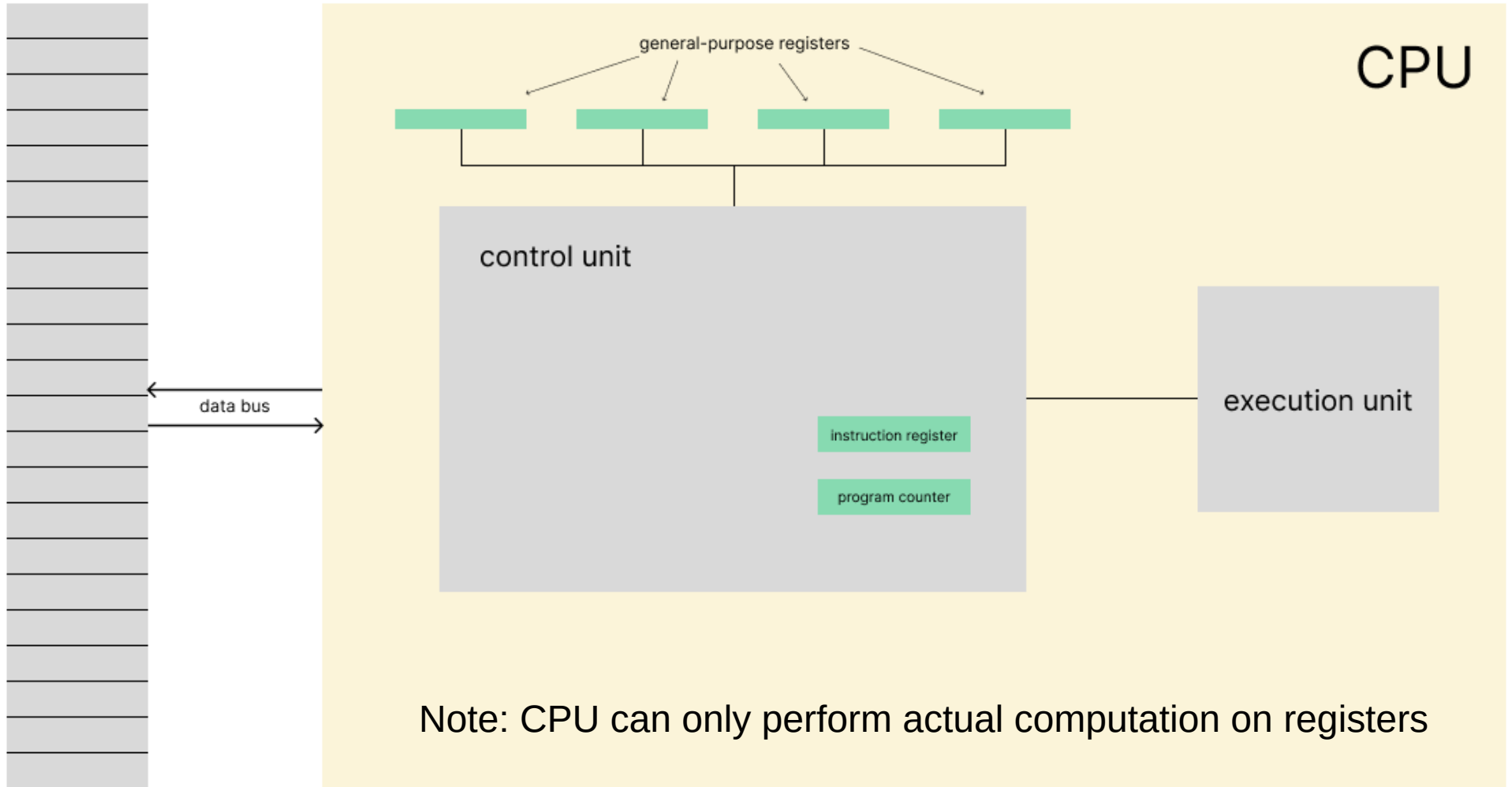
Quiz 1 Format

- 60 minutes
- Will involve MCQs, short questions, and code writing
 - Make sure you are able to read, write and understand Python codes, especially those that make use of the `threading` module
 - Same for SQL statements
- No questions on Git/GitHub

Lectures 3-7



main memory



Note: CPU can only perform actual computation on registers

How a CPU executes code

LOAD R0 0x00
LOAD R1 0xA0
ADD R0 R1

- Fetch, decode, execute
 - Fetch: CPU loads instruction into the memory (in binary)
 - Decode: CPU interprets the instruction
 - Execute: Execution unit carries out the interpreted instruction
- Notice how the three stages involve different parts of the CPU
- Question: This property enables which aspect of CPU optimization?

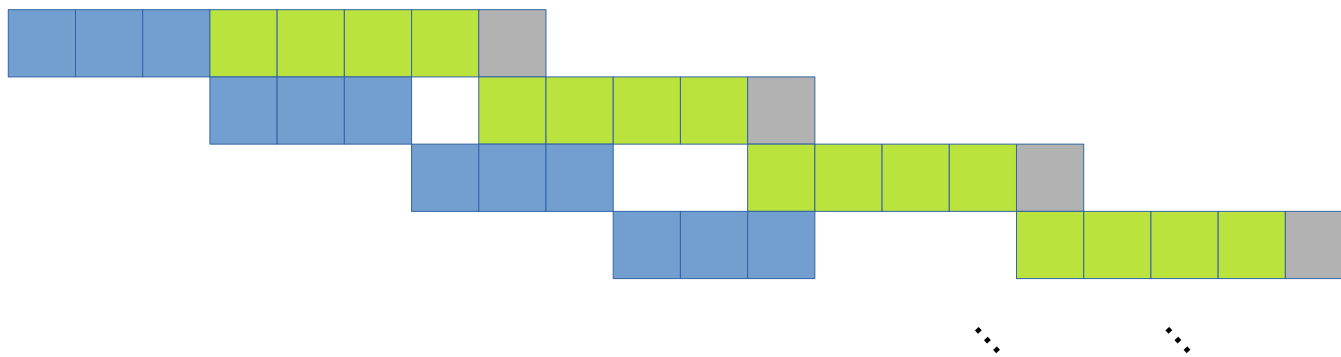
Purpose of a program: modify CPU's <u>state</u>

Threads vs processes

- Process
 - A program in execution (along with relevant contexts)
 - Different processes have mutually isolated address spaces
 - Heavier weight when creating and switching between processes
- Threads
 - Smallest unit of execution within a process (all processes have a “main” thread)
 - Threads within a process share address spaces***
But they still own their own contexts
 - Lighter weight when creating and switching between threads

Memory latency vs bandwidth

- Latency
 - How much time it takes for data to arrive at CPU after initiating request
 - Has unit of time (e.g. ns)
- Bandwidth
 - Rate at which data arrives after initial latency
 - Has unit of byte/second

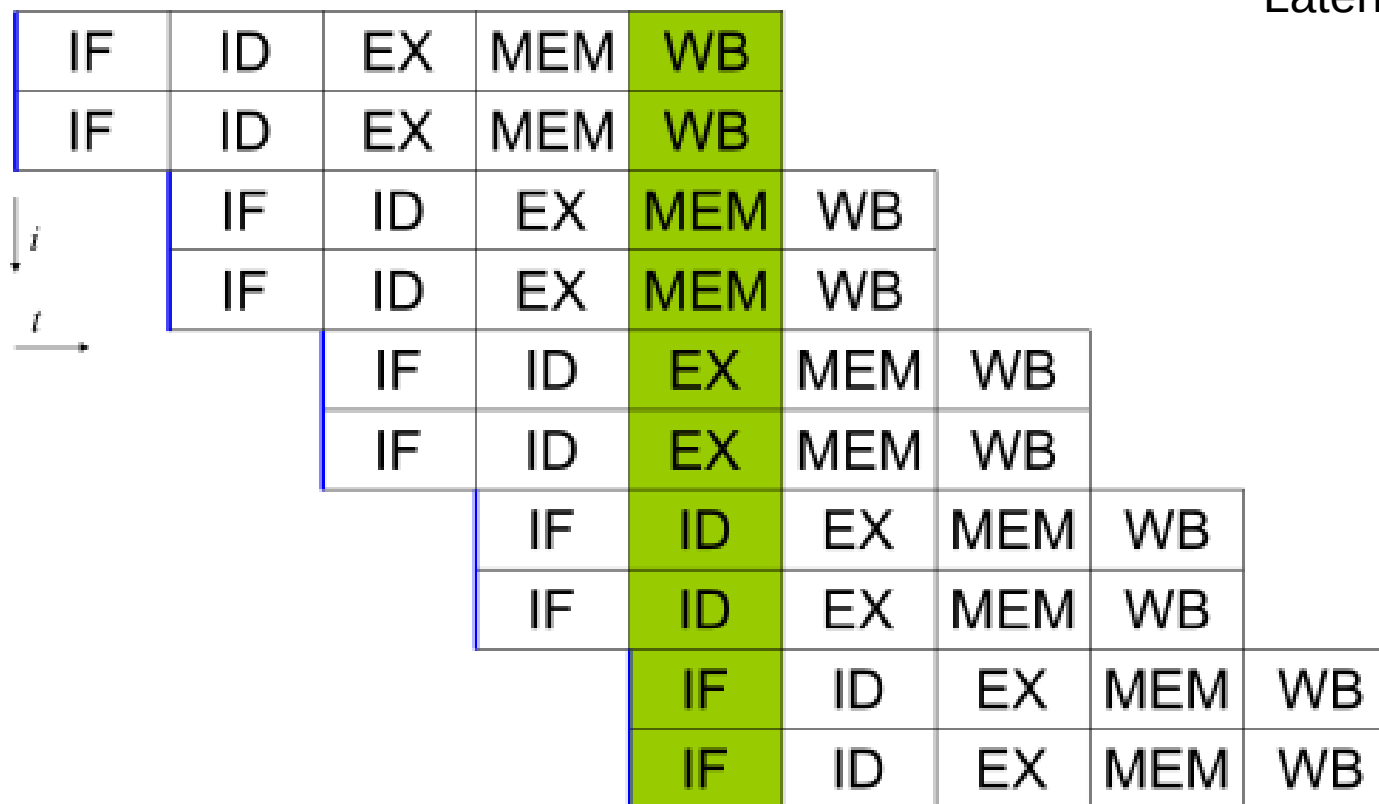


CPU Optimization

- Typically beyond programmer's control
 - Pipelining
 - Caches
 - Branch prediction
 - Out-of-order execution
 - Superscalarity
 - Simultaneous multithreading (SMT)
- Typically within programmer's control
 - SIMD

Pipelining

Latency hiding!



Caches

- Small, on-chip memory for quick access
- Loads data from main memory in “batches” – cache lines
- If data present in cache when requested: cache hit, else cache miss
 - Cache misses mean performance penalty!

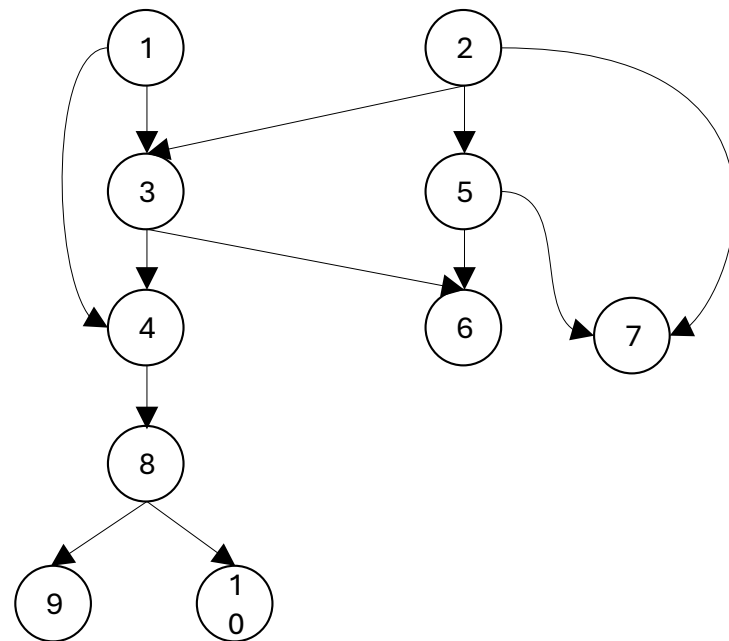
Superscalarity & out-of-order execution

- Based on ILP, multiple instructions per clock cycle
- Superscalar core: multiple execution units

```
1  a=1
2  b=2

3  c=a+b
4  d=a+c
5  e=2*b
6  f=c+e
7  g=e+b

8  if d>3:
9      print("Hello")
   else:
10     print("World")
```



SMT

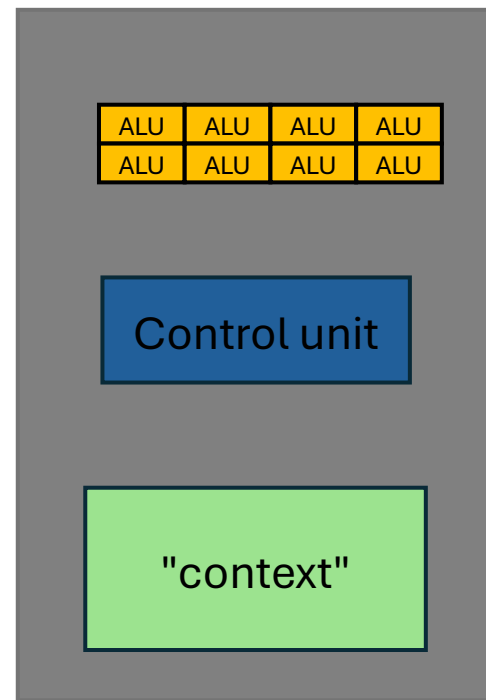
- Thread-level latency hiding
- Need to store different threads' execution contexts
- Different threads still share same execution resources
- Typically boost performance by 20-30% (why not double?)



Trade-off:
Latency of one thread increases, but
overall throughput is increased

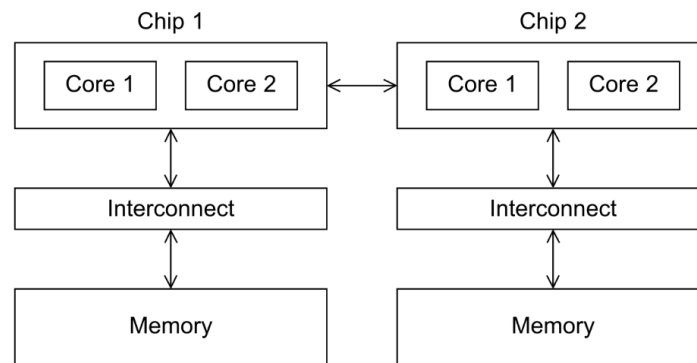
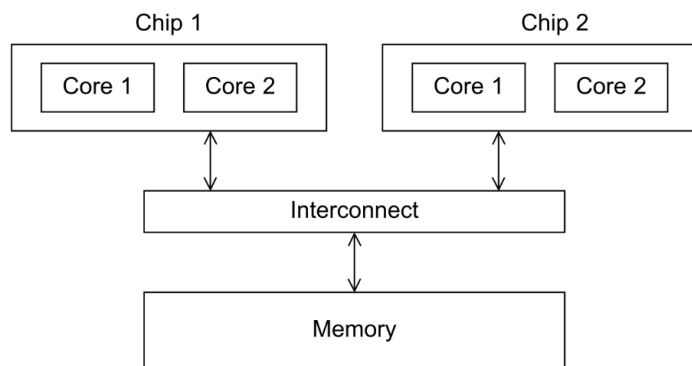
SIMD

- Single CPU instruction acts on multiple data items
number of items = “width” of SIMD
- Requires “vector-like” execution units and registers
- Does not mesh well with branching
- In the picture, peak FLOPs per clock cycle?

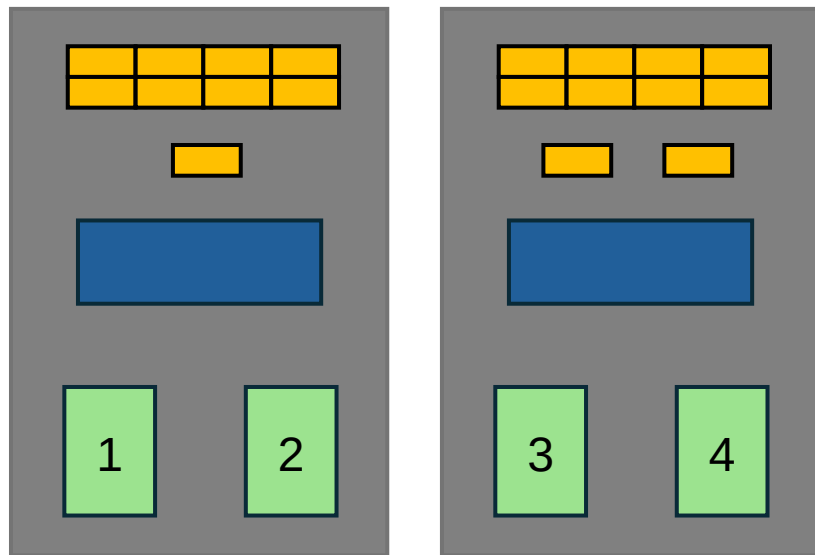


Multi-core architecture

- Have different memory systems (why?)
 - UMA
 - NUMA



An example



- Dual core
- Superscalar
- Two-way multithreaded
- 8-wide SIMDs

Thread 1
...
VMUL VR0 VR1
ADD R2 R3
...

Thread 2
...
VMUL VR0 VR1
VADD VR0 VR2
...

Thread 3
...
MUL R0 R1
ADD R1 R3
...

Thread 4
...
VADD VR2 VR3
ADD R5 R7
...

GPU

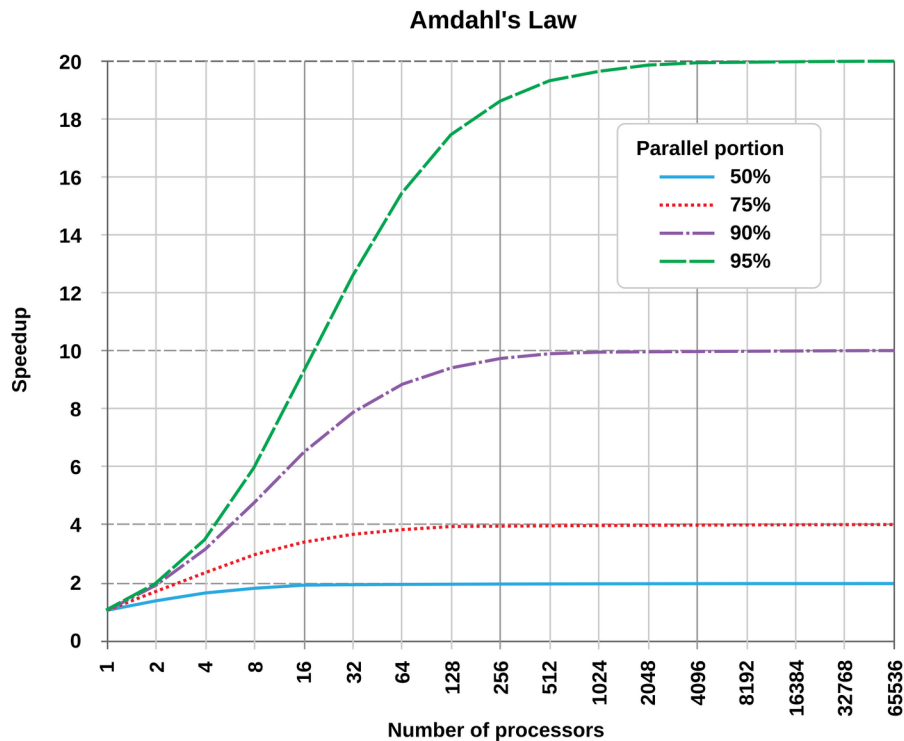
- Basically large-scale SIMD with high memory bandwidth
 - At the cost of each core being less sophisticated – higher latency
- Has lots of execution contexts – great latency hiding ability!
 - This is assuming you do have enough work to do in the first place...

Parallel programming recipe

1. Decomposition into independent sub-problems
2. Assigning work to workers
3. Orchestration (i.e. communication)
4. Mapping workers to actual hardware execution units
 - Should the workers be threads, processes, nodes in a cluster, or something else? What determines this choice?

Upper bound on speed-up in parallelized program

- Amdahl's law

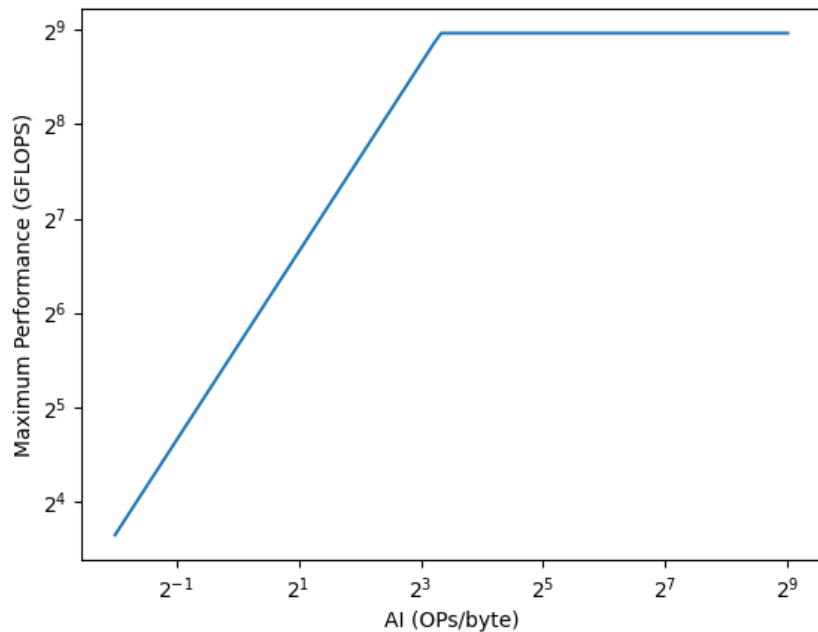


$$S \leq \frac{1}{1 - f + f/N}$$

compute-bound vs memory-bound

- Roofline model

Real Performance $\leq \min(\text{Peak Performance}, \text{AI} \times \text{Peak Memory Bandwidth})$



Shared-memory parallel programming in Python

- Available from Python 3.13(t) onwards
- `threading` module, `concurrent.futures()` module

```
import threading

def worker(data):
    # work on data, may involve synchronization

threads = []

# create and start threads
for data in items():
    # create threads
    t = threading.Thread(target=worker, args=(data,))
    threads.append(t)
    t.start()

# wait for threads to finish
for t in threads:
    t.join()
```

```
from concurrent.futures import ThreadPoolExecutor

def worker(data):
    result = process(data)
    return result

with ThreadPoolExecutor(max_workers=num_threads) as executor:
    # data_chunks is a list of chunks of data
    results = list(executor.map(worker, data_chunks))
```

Pitfalls of shared-memory systems

- Race condition:
program results depend on (non-deterministic) ordering of worker executions
 - Classic example: non-atomic operations (e.g. +=)
- Deadlock:
Program unable to proceed since different threads “block” each other

Synchronization primitives

- Lock
- Semaphore
 - Typically used in problems which involve resource counting (e.g. producer-consumer)
- Condition variable
- Barrier

They are to avoid **race conditions**!

Useful resources

- “An Introduction to Parallel Programming” by Peter Pacheco
Chapters 1, 2, 4 and 6
- “The Art of Multiprocessor Programming” by Herlihy & Shavit
 - In-depth explanation of various synchronization primitives and how they actually work, probably too detailed for this course

Some review questions

Caches

- Questions
 - What are caches?
 - Why do we need caches? Why do they work?
 - What are “cache hits” and “cache misses”?
 - What is a cache line? Why is it a thing?
 - What is a cache line eviction? Why does it occur?
 - Consider traversing an n -by- n matrix of floats. Our cache has two cache lines, each of which are 16 bytes in size. How many cache hits?

Branch prediction

- Questions
 - What are branch predictions? How do they help CPU execute codes faster?
 - Why are there performance penalties in branch mis-predictions?

Out-of-order execution

- Questions
 - When are CPUs allowed to execute out-of-order?
 - Why does it help in terms of performance?

Superscalarity

- Questions
 - What does it mean for a core to be, say, 4-way superscalar?
 - Does superscalarity always increase performance? What kind of programs would not benefit much from having superscalar cores?

SMT

- Questions
 - Why do we have SMT? How does it help in terms of performance?
 - What hardware components need to be duplicated to achieve SMT?
 - Is it true that “more threads = better”?

SIMD

- Questions
 - What does it mean when I say a SIMD unit is 16-wide?
 - How does SIMD differ from SMT?
 - What happens when I have branches when executing SIMD codes?

Multi-core architecture

- Questions
 - What led to the rise of multi-core architecture in the early 2000's? Why couldn't we just keep increasing CPU frequency?
 - How can we take advantage of multi-core architectures? Can we just reuse the same code for single-core computers?
 - Why don't we just keep increasing number of cores?

GPU vs CPU

- Questions
 - What are the major differences between GPUs and CPUs?
 - What kind of tasks would be good to run on GPUs?
 - What kind of tasks would be good to run on CPUs?
 - Comment on the following statement: “we should always compute dot products using GPUs since GPUs have many more execution units than CPU”

Python and GIL

- Questions
 - What is GIL in Python? Why is it there?
 - Until which version of Python can we finally turn off the GIL?
 - How does the GIL affect multithreading performance?

Latency vs Bandwidth

- Questions
 - Explain the difference between latency and bandwidth
 - Give examples of programs/applications that are (i) latency bound, and (ii) bandwidth-bound

Amdahl's law

- Questions
 - Explain the set-up of Amdahl's law and derive it
 - Why does Amdahl's law only give an upper bound?

Parallel programming recipe

- Questions
 - Explain what load imbalance is, and how it affects performance of a parallel program
 - Give some examples of algorithms based on divide-and-conquer, and show how to parallelize them.

Synchronization primitives

- Questions
 - Explain difference(s) between condition variables and semaphores
 - Give examples for how to use each of the primitives we introduced
 - Give an example in which the program will hit a deadlock
 - Identify what's wrong:

```
import threading

counter = 0

def increment():
    global counter
    for _ in range(100000):
        counter += 1

threads = []
for _ in range(10):
    t = threading.Thread(target=increment)
    threads.append(t)
    t.start()

for t in threads:
    t.join()
```